BÁO CÁO PROJECT K8S

TRIỂN KHAI HỆ THỐNG AN NINH HẠ TẦNG MICROSERVICES TRÊN NỀN TẢNG CLOUD-NATIVE

Upstream Kubernetes, Cilium eBPF CNI, Dual-Gateway Suricata IDS/IPS, Rancher Management & Wazuh SIEM

Mục lục

1 Kiến trúc hệ thống **1**

1.1 Tổng quan kiến trúc . . . 1
1.2 Các service ứng dụng triển khai . . . 2
1.3 Phân vùng mạng và định tuyến . . . 3
1.3.1 Bảng định tuyến . . . 3
1.4 Security Groups và kiểm soát truy cập . . . 4
1.5 Các port và cơ chế chuyển tiếp . . . 5
1.6 Mạng Pod Cilium eBPF và Hubble Observability . . . 6
1.7 Hạ tầng máy chủ ảo EC2 . . . 7

2 Các thành phần triển khai **8**

2.1 Bảng tổng hợp các thành phần hệ thống . . . 10
2.2 Chi tiết từng thành phần . . . 11
2.2.1 High Availability Edge Gateway . . . 11
2.2.2 Bộ cân bằng tải HAProxy Layer 4 . . . 11
2.2.3 Suricata IDS/IPS . . . 11
2.2.4 Kubernetes Control Plane và Workers . . . 11
2.2.5 Cilium CNI eBPF và Hubble Observability . . . 11
2.2.6 Ingress-NGINX Controller . . . 12
2.2.7 Rancher Management Server . . . 12
2.2.8 Wazuh SIEM & XDR . . . 12
2.2.9 Falco Cloud-Native Runtime Security và Falcosidekick UI . . . 12
2.2.10 Google Online Boutique . . . 12
2.2.11 Horizontal Pod Autoscaler HPA v2 và Metrics-Server . . . 13
2.2.12 Zero-Trust Network Policies . . . 13

3 Triển khai các thành phần **14**

3.1 Cấu hình Gateway 1 & Gateway 2 . . . 14
3.1.1 Định tuyến Dual-ENI và Policy Routing . . . 14
3.1.2 Cấu hình chuyển tiếp gói tin và tường lửa IPTables NAT . . . 15
3.1.3 Cấu hình bộ cân bằng tải HAProxy Layer 4 Active/Backup Cross-AZ 15
3.1.4 Cấu hình hệ thống phát hiện/ngăn chặn xâm nhập Suricata DPI . . . 18
3.2 Khởi tạo cụm Kubernetes và K8s API Auditing . . . 19
3.2.1 Khởi tạo Control Plane và tham số Kube-apiserver Auditing . . . 19
3.2.2 Kubernetes Audit Policy . . . 20

3.3 Triển khai Phân hệ Mạng CNI eBPF Cilium và Hubble Observability . . . 21
3.3.1 Cài đặt Cilium Kernel Mode và cấu hình mạng Overlay VXLAN . . 21
3.3.2 Kích hoạt Hubble Relay và công khai giao diện Hubble UI 21
3.4 Triển khai Ingress-NGINX Multi-AZ và Rancher Management Server . . . 22
3.4.1 Kiến trúc Ingress-NGINX 22
3.4.2 Triển khai Rancher Management Server 23
3.5 Triển khai Ứng dụng Google Online Boutique và Chính sách Zero-Trust . . 23
3.5.1 Các Microservices của ứng dụng Online Boutique 23
3.5.2 Zero-Trust network policy (k8s/04-network-policies.yaml) 23
3.6 Triển khai Wazuh SIEM 25
3.6.1 Kiến trúc và Luồng tiếp nhận Log trong Wazuh SIEM 25
3.6.2 Cấu hình thu thập log trên các máy chủ Agent (/var/ossec/etc/ossec.conf) 26
3.6.3 Cấu hình tiếp nhận kết nối trên Wazuh Manager (/var/ossec/etc/ossec.conf) 26
3.6.4 Tập luật (/var/ossec/etc/rules/local$_rules.xml$) 27
3.7 Triển khai Falco Container Runtime Security & Falcosidekick UI 29
3.7.1 Cấu hình DaemonSet Falco và chuyển tiếp sự kiện (/etc/falco/falco.yaml) 29
3.7.2 Cấu hình Falcosidekick và Falcosidekick Web UI 29
3.7.3 Bộ luật giám sát runtime 30

4 Kiểm thử hệ thống 32
4.1 Kiểm thử 1: Truy cập các cổng dịch vụ qua AWS NLB 32
4.1.1 Kịch bản kiểm thử 32
4.1.2 Kết quả kiểm thử 32
4.2 Kiểm thử 2: Phát hiện tấn công Web qua Suricata IDS/IPS 33
4.2.1 Kịch bản kiểm thử 33
4.2.2 Kết quả kiểm thử 33
4.3 Kiểm thử 3: Giám sát tập trung trên Wazuh SIEM 34
4.3.1 Kịch bản kiểm thử 34
4.3.2 Kết quả kiểm thử 34
4.4 Kiểm thử 4: Quan sát luồng mạng thời gian thực qua Cilium Hubble . . . 35
4.4.1 Kịch bản kiểm thử 35
4.4.2 Kết quả kiểm thử 35
4.5 Kiểm thử 5: Cô lập mạng nội bộ Zero-Trust 36
4.5.1 Kịch bản kiểm thử 36
4.5.2 Kết quả kiểm thử 36
4.6 Kiểm thử 6: Co giãn tự động HPA và quản lý tài nguyên 37
4.6.1 Kịch bản kiểm thử 37
4.6.2 Kết quả kiểm thử 37
4.7 Kiểm thử 7: Cân bằng tải và chuyển đổi dự phòng c 38

4.7.1 Kịch bản kiểm thử 38
4.7.2 Kết quả kiểm thử 38
4.8 Kiểm thử 8: Giám sát an ninh Runtime Container với Falco và Falcosidekick UI 41
4.8.1 Kịch bản kiểm thử 41
4.8.2 Kết quả kiểm thử 41

Danh sách hình vẽ

1.1 Sơ đồ kiến trúc tổng thể hệ thống an ninh hạ tầng Microservices trên AWS 1

Danh sách bảng

1.1 Bảng tóm tắt các service ứng dụng Online Boutique triển khai trên cụm K8s 2
1.2 Bảng phân bổ không gian địa chỉ IP và Subnet trên AWS 3
1.3 Bảng phân bổ port và cơ chế chuyển tiếp lưu lượng 5
1.4 Bảng cấu hình máy chủ EC2 trong hệ thống 7

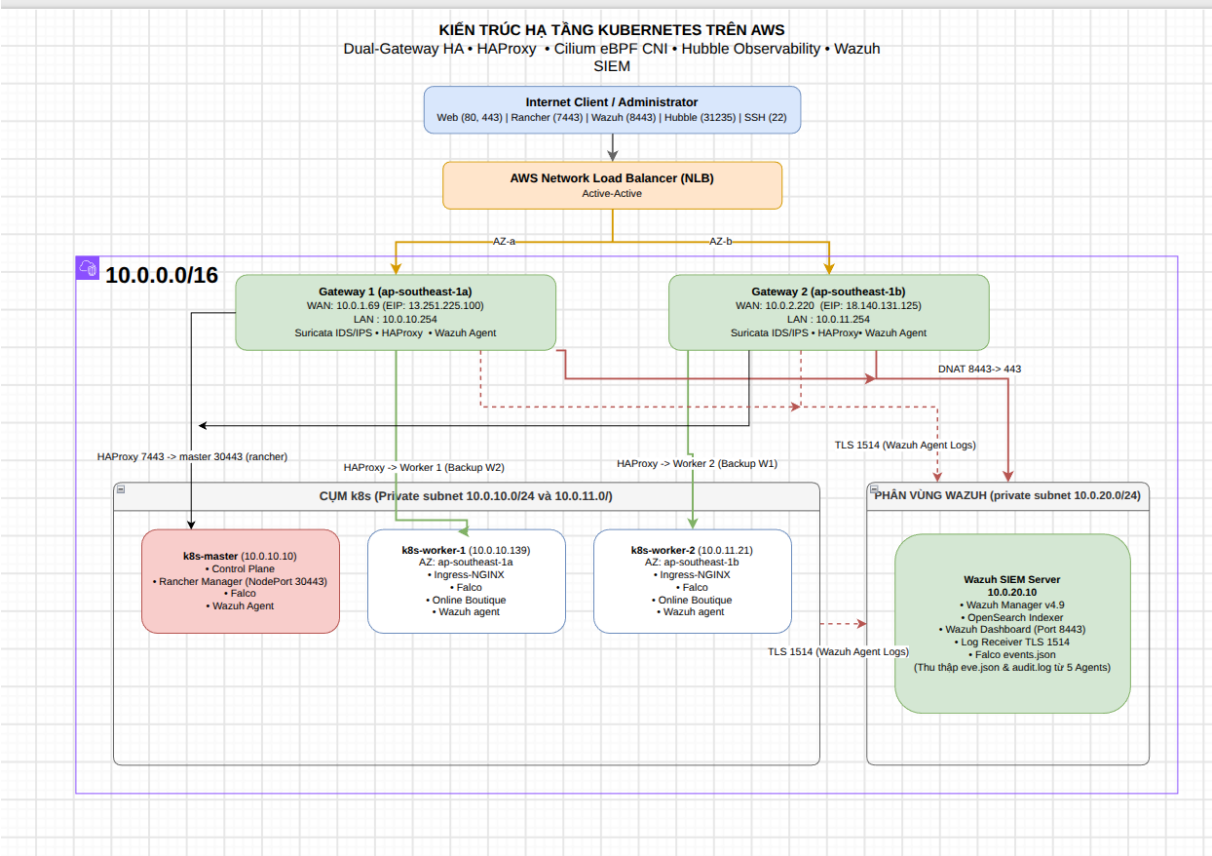
2.1 Tổng hợp các thành phần công nghệ trong hệ thống 10

4.1 Do giãn của Pods theo mức độ tải CPU 37

1. Kiến trúc hệ thống

1.1 Tổng quan kiến trúc

Hệ thống được triển khai trên AWS. Kiến trúc đáp ứng tính sẵn sàng cao trên 2 Availability Zones ap-southeast-1a và ap-southeast-1b, phân tách giữa mạng biên tiếp nhận lưu lượng Internet, vùng mạng nội bộ cụm Kubernetes và vùng quản trị an ninh tập trung.



Hình 1.1: Sơ đồ kiến trúc tổng thể hệ thống an ninh hạ tầng Microservices trên AWS

Lưu lượng Internet đi vào được tiếp nhận bởi AWS Network Load Balancer phân bổ tải tới Gateway 1 và Gateway 2. Tại đây, gói tin được kiểm tra qua Suricata IDS/IPS trước khi chuyển tiếp qua HAProxy vào cụm Kubernetes nội bộ. Bên trong cụm Kubernetes, toàn bộ các Pod và Container được bảo vệ và giám sát an ninh runtime thời gian thực bằng Falco eBPF.

1.2 Các service ứng dụng triển khai

Ứng dụng thương mại điện tử Google Online Boutique gồm 11 microservices kết hợp cùng service sinh tải `loadgenerator`. Các service được đóng gói dưới dạng container, vận hành trong namespace `online-boutique` trên cụm Kubernetes:

Bảng 1.1: Bảng tóm tắt các service ứng dụng Online Boutique triển khai trên cụm K8s

Service	**Ngôn ngữ**	**Cổng**	**Giao thức**	**Vai trò chức năng**
`frontend`	Go	80	HTTP	Web UI, hiển thị giao diện và sản phẩm cho người dùng
`cartservice`	C# .NET	7070	gRPC	Quản lý giỏ hàng của từng phiên người dùng
`redis-cart`	Redis	6379	TCP	In-memory cache lưu trữ dữ liệu giỏ hàng
`productcatalogservice`	Go	3550	gRPC	Cung cấp danh mục sản phẩm và tìm kiếm món đồ
`checkoutservice`	Go	5050	gRPC	Tiếp nhận đơn hàng, điều phối thanh toán và vận chuyển
`paymentservice`	Node.js	50051	gRPC	Xử lý thanh toán thẻ tín dụng, bảo vệ bởi Zero-Trust Policy
`currencyservice`	Node.js	7000	gRPC	Chuyển đổi và tính toán tỷ giá ngoại tệ
`emailservice`	Python	8080	gRPC	Soạn thảo và gửi email xác nhận đặt hàng
`shippingservice`	Go	50051	gRPC	Tính cước phí vận chuyển và cấp mã vận đơn
`recommendationservice`	Python	8080	gRPC	Đề xuất gợi ý sản phẩm liên quan theo ngữ cảnh
`adservice`	Java	9555	gRPC	Hiển thị thông tin quảng cáo và khuyến mãi
`loadgenerator`	Python	–	HTTP	Sinh tải liên tục theo hành vi người dùng để kích hoạt HPA

1.3 Phân vùng mạng và định tuyến

Toàn bộ hạ tầng mạng được nằm trong Amazon VPC `ncs-project-vpc` dải mạng `10.0.0.0/16` thành 5 Subnet độc lập trên 2 Availability Zones:

Bảng 1.2: Bảng phân bổ không gian địa chỉ IP và Subnet trên AWS

Tên Subnet	Vùng AZ	Dải CIDR	Loại mạng	Mục đích sử dụng
`public-wan-a`	`ap-southeast-1a`	`10.0.1.0/24`	Public	Gateway 1 WAN interface và AWS NLB Node 1
`public-wan-b`	`ap-southeast-1b`	`10.0.2.0/24`	Public	Gateway 2 WAN interface và AWS NLB Node 2
`private-k8s-a`	`ap-southeast-1a`	`10.0.10.0/24`	Private	K8s Master `10.0.10.10` và K8s Worker 1 `10.0.10.139`
`private-k8s-b`	`ap-southeast-1b`	`10.0.11.0/24`	Private	K8s Worker 2 `10.0.11.21`
`mgmt-wazuh`	`ap-southeast-1a`	`10.0.20.0/24`	Private	Wazuh SIEM Server `10.0.20.10`

1.3.1 Bảng định tuyến

1. **Public Route Table**: Liên kết với 2 Public Subnet `public-wan-a` và `public-wan-b`. Quy tắc `0.0.0.0/0` hướng ra Internet Gateway `igw`, cho phép Gateway và NLB tiếp nhận lưu lượng Internet qua IP `13.251.225.100` và `18.140.131.125`.

2. **Private Route Table A**: Gắn với `private-k8s-a`. Quy tắc `0.0.0.0/0` trỏ tới card LAN `ens6` IP `10.0.10.254` của Gateway 1. Toàn bộ lưu lượng Egress từ Master và Worker 1 đều đi qua Gateway 1.

3. **Private Route Table B**: Gắn với `private-k8s-b`. Quy tắc `0.0.0.0/0` trỏ tới card LAN `ens6` IP `10.0.11.254` của Gateway 2, đảm bảo phân vùng AZ-b duy trì luồng Egress độc lập.

4. **Management Route Table**: Gắn với `mgmt-wazuh`. Quy tắc `0.0.0.0/0` trỏ tới LAN Gateway 1 IP `10.0.10.254` để cập nhật các gói an ninh từ Internet.

1.4 Security Groups và kiểm soát truy cập

- **ncs-project-sg-gateway**: Áp dụng cho các máy chủ Edge Gateway. Mở các cổng công khai tiếp nhận từ Internet: cổng 80 (HTTP), cổng 443 (HTTPS), cổng 7443 (Rancher Dashboard), cổng 8443 (Wazuh Dashboard), cổng 8404 (HAProxy Stats) và cổng 22 (SSH).
- **ncs-project-sg-k8s**: Áp dụng cho K8s Master và Worker nodes. Chặn truy cập trực tiếp từ Internet. Chỉ chấp nhận lưu lượng chuyển tiếp từ card LAN của Gateway `10.0.10.254`, `10.0.11.254` và cho phép thông suốt lưu lượng nội bộ giữa các Node trong dải `10.0.0.0/16`.
- **ncs-project-sg-wazuh**: Áp dụng cho máy chủ Wazuh SIEM. Mở cổng 1514 (Wazuh Agent Events) và 1515 (Agent Registration) cho toàn bộ dải mạng VPC `10.0.0.0/16`; mở cổng 443 và 55000 cho các kết nối chuyển tiếp từ Gateway LAN.

1.5 Các port và cơ chế chuyển tiếp

Bảng 1.3: Bảng phân bổ port và cơ chế chuyển tiếp lưu lượng

Tên dịch vụ	Cổng ngoài	Giao thức	Đích chuyển tiếp	Mô tả
Online Boutique HTTP	**80**	HTTP	`10.0.10.139` / `10.0.11.21`	HAProxy L4 chuyển tiếp Ingress trên Worker (Active/Backup
Online Boutique HTTPS	**443**	HTTPS	`10.0.10.139` / `10.0.11.21`	HAProxy L4 chuyển tiếp Ingress trên Worker (Active/Backup
Rancher Server	**7443**	HTTPS	`10.0.10.10:30443`	HAProxy L4 chuyển tiếp Rancher NodePort trên Master
Wazuh SIEM Dashboard	**8443**	HTTPS	`10.0.20.10:443`	IPTables DNAT chuyển tiếp tới máy chủ SIEM SOC
HAProxy Stats	**8404**	HTTP	Trực tiếp tại Gateway	Thống kê phiên kết

1.6 Mạng Pod Cilium eBPF và Hubble Observability

Cụm Kubernetes triển khai Cilium v1.16.2 hoạt động ở chế độ eBPF Kernel Mode:

- Các Pod trên các Node giao tiếp với nhau qua đường truyền đóng gói VXLAN tunnel trên giao thức UDP cổng 8472. Dải mạng Pod nội bộ được quy hoạch trong không gian `10.244.0.0/16`.
- Tích hợp Hubble Relay và Hubble UI, cho phép theo dõi trực quan luồng dữ liệu thời gian thực.

1.7 Hạ tầng máy chủ ảo EC2

Bảng 1.4: Bảng cấu hình máy chủ EC2 trong hệ thống

Máy chủ	**Loại Instance**	**vCPU**	**RAM**	**Ổ đĩa EBS**	**Nhiệm vụ chính**
`pfsense-gateway-1`	`t3.small`	2	2 GB	20 GB gp3	Gateway 1 AZ-a, Suricata IDS/IPS, HAProxy L4, NAT, Bastion
`pfsense-gateway-2`	`t3.small`	2	2 GB	20 GB gp3	Gateway 2 AZ-b, Suricata IDS/IPS, HAProxy L4
`k8s-master`	`c7i-flex.large`	2	4 GB	20 GB gp3	K8s Control Plane, Rancher Server
`k8s-worker-1`	`c7i-flex.large`	2	4 GB	20 GB gp3	K8s Worker node 1 AZ-a, Ingress-NGINX
`k8s-worker-2`	`c7i-flex.large`	2	4 GB	20 GB gp3	K8s Worker node 2 AZ-b, Ingress-NGINX
`wazuh-siem`	`t3.large`	2	8 GB	50 GB gp3	Dedicated SIEM Server, OpenSearch Indexer, SOC

2. Các thành phần triển khai

2.1 Bảng tổng hợp các thành phần hệ thống

Bảng 2.1: Tổng hợp các thành phần công nghệ trong hệ thống

Thành phần	Công nghệ	Vai trò chức năng
High Availability Gateway	Dual-ENI Linux + AWS NLB	Cân bằng tải Active-Active, định tuyến NAT, SSH Bastion
HAProxy	HAProxy v2.8+ TCP Stream	Cân bằng tải, TCP Health Check
Suricata IDS/IPS	Suricata af-packet	Bắt gói tin tầng biên, kiểm tra sâu DPI, phát hiện tấn công Web
Kubernetes	Upstream K8s v1.31.14	Nền tảng điều phối container, multi-AZ
Cilium eBPF CNI	Cilium v1.16.2 Kernel Mode	Mạng Pod, VXLAN tunnel
Hubble Observability	Hubble Relay & Hubble UI	Quan sát luồng mạng thời gian thực L3, L4, L7
Ingress-NGINX	Ingress Controller hostNetwork	Điểm vào Web tập trung, kết thúc TLS, Virtual Host, rate limiting
Rancher Server	SUSE Rancher v2.9	Quản trị K8s đồ họa qua Web, giám sát tài nguyên
Wazuh SIEM & XDR	Wazuh Server & Indexer v4.9	Trung tâm SOC, OpenSearch Indexer, tương quan cảnh báo an ninh
Wazuh Agents	Wazuh Agent trên 5 máy chủ	Thu thập log Suricata eve.json, K8s audit.log và bảo mật OS
Online Boutique	Google Demo 12 Services	Ứng dụng thương mại điện tử đa ngôn ngữ microservices
Metrics-Server & HPA	K8s HPA v2	Thu thập metrics Kubelet, tự động co giãn Pod theo tải CPU
Network Policies	Zero-Trust & Cilium NetPol	Cô lập dịch vụ nội bộ
Falco Runtime Security	Falco modern_ebpf	Giám sát lời gọi hệ thống (Syscalls) bên trong container theo thời gian thực

2.2 Chi tiết từng thành phần

2.2.1 High Availability Edge Gateway

Cụm Edge Gateway gồm 2 máy chủ Ubuntu Linux kết hợp AWS Network Load Balancer. Mỗi máy chủ được cấu hình 2 card mạng: card WAN `ens5` gắn Elastic IP công khai và card LAN `ens6` kết nối vào mạng nội bộ. Gateway kết hợp bộ cân bằng tải HAProxy Layer 4 để chuyển tiếp thông minh các cổng dịch vụ 80 (HTTP), 443 (HTTPS), 7443 (Rancher), và duy trì IPTables DNAT cho cổng 8443 (Wazuh SIEM), SNAT Masquerade cho luồng Egress và đóng vai trò SSH Bastion qua SSH Agent Forwarding.

2.2.2 Bộ cân bằng tải HAProxy Layer 4

HAProxy được triển khai trên cả hai máy chủ Edge Gateway hoạt động ở chế độ Layer 4 TCP Stream (`mode tcp`). HAProxy đóng vai trò reverse proxy trung gian giữa AWS NLB và các Ingress Pod trên các K8s Worker nodes:

- **Cơ chế Active/Backup chéo vùng**: Gateway 1 ưu tiên chuyển tiếp tới Worker 1 (`10.0.10.139`) và dự phòng Worker 2 (`10.0.11.21`); Gateway 2 ưu tiên Worker 2 và dự phòng Worker 1.

2.2.3 Suricata IDS/IPS

Suricata hoạt động ở chế độ `af-packet` trên card WAN `ens5`, thực hiện Deep Packet Inspection để phát hiện các cuộc tấn công Web tầng biên.Nhận diện các dạng tấn công: SQL Injection, Cross-Site Scripting, Directory Traversal và thăm dò tập tin nhạy cảm `.env`, `.git`. Logs được xuất tại `/var/log/suricata/eve.json` để chuyển về Wazuh SIEM.

2.2.4 Kubernetes Control Plane và Workers

Cụm Kubernetes sử dụng Containerd runtime SystemdCgroup. Hai Worker node phân bổ trên 2 Availability Zones ap-southeast-1a và ap-southeast-1b để vận hành các microservices và Ingress-NGINX.

2.2.5 Cilium CNI eBPF và Hubble Observability

Cilium v1.16.2 hoạt động ở chế độ eBPF Kernel Mode. Mạng Pod giao tiếp qua VXLAN cổng 8472/UDP trên dải CIDR `10.244.0.0/16`.

2.2.6 Ingress-NGINX Controller

Ingress-NGINX được nâng cấp triển khai trên cả 2 Worker node (Worker 1 và Worker 2) với `hostNetwork: true`, tiếp nhận trực tiếp lưu lượng từ HAProxy tại cổng 80 (HTTP) và cổng 443 (HTTPS). Controller tiếp nhận và điều phối lưu lượng trực tiếp tới các Service trong cụm, áp dụng rate limiting 50 req/s.

2.2.7 Rancher Management Server

Rancher Server v2.9 được triển khai trong namespace `cattle-system`, cung cấp giao diện web trực quan để quản trị cụm, phân quyền RBAC và giám sát tài nguyên. Rancher lắng nghe trên Service NodePort `30443` và được Gateway chuyển tiếp qua cổng công khai 7443 (HTTPS).

2.2.8 Wazuh SIEM & XDR

Wazuh SIEM chạy trong Management Subnet, bao gồm Wazuh Manager, OpenSearch Indexer và Wazuh Dashboard cổng 8443 (HTTPS). Năm Wazuh Agents được cài đặt trên Gateway 1, Gateway 2, Master, Worker 1 và Worker 2 để thu thập log Suricata `eve.json`, K8s API Server `audit.log`.

2.2.9 Falco Cloud-Native Runtime Security và Falcosidekick UI

Kiến trúc phân tầng của Falco

Falco hoạt động đồng thời trên hai phân vùng:

- **Phân vùng Kernel**: Giám sát mọi lời gọi hệ thống (system calls - syscalls) phát sinh từ các tiến trình bên trong container
- **Phân vùng Người dùng**: Daemon Falco liên tục đọc dữ liệu sự kiện và đưa vào bộ máy đối sánh luật để phân tích hành vi bất thường theo thời gian thực.

2.2.10 Google Online Boutique

Ứng dụng thương mại điện tử gồm 12 microservices triển khai trong namespace `online-boutique`: `frontend`, `cartservice`, `redis-cart`, `productcatalogservice`, `checkoutservice`, `paymentservice`, `currencyservice`, `emailservice`, `shippingservice`, `recommendationservice`, `adservice` và `loadgenerator`.

2.2.11 Horizontal Pod Autoscaler HPA v2 và Metrics-Server

Metrics-Server v0.7.2 thu thập số liệu CPU và bộ nhớ từ Kubelet qua cổng 10250. HPA v2 tự động co giãn số lượng Pod của các dịch vụ cốt lõi `frontend`, `cartservice`, `checkoutservice` khi mức sử dụng CPU vượt ngưỡng 70%–75%.

2.2.12 Zero-Trust Network Policies

Hệ thống thiết lập mô hình Zero-Trust ở tầng mạng Pod: cô lập hoàn toàn `paymentservice`, chỉ chấp nhận kết nối từ `checkoutservice` qua cổng gRPC 50051; áp dụng `CiliumNetworkPolicy` cho phép Ingress host traffic tới `frontend`,...

3. Triển khai các thành phần

3.1 Cấu hình Gateway 1 & Gateway 2

Hệ thống biên mạng được thiết kế theo mô hình HA đa vùng (Multi-AZ), bao gồm hai máy chủ Gateway độc lập: Gateway 1 đặt tại Availability Zone ap-southeast-1a (card WAN có IP nội bộ `10.0.1.69` gắn Elastic IP công khai `13.251.225.100`, card LAN nội bộ `10.0.10.254`) và Gateway 2 đặt tại ap-southeast-1b (card WAN có IP nội bộ `10.0.2.220` gắn Elastic IP công khai `18.140.131.125`, card LAN nội bộ `10.0.11.254`).

3.1.1 Định tuyến Dual-ENI và Policy Routing

Mỗi máy chủ Gateway có 2 card mạng độc lập: card `ens5` thuộc Public Subnet (WAN) tiếp nhận lưu lượng Internet và card `ens6` thuộc Private Subnet (LAN) kết nối vào mạng nội bộ của cụm máy chủ. Để ngăn chặn Asymmetric Routing khiến các kết nối TCP bị ngắt quãng, cấu hình Netplan phân chia rõ ràng metric định tuyến giữa hai giao diện:

```
network:
  version: 2
  renderer: networkd
  ethernets:
    ens5:
      dhcp4: true
      dhcp4-overrides:
        route-metric: 100
    ens6:
      dhcp4: false
      addresses:
        - 10.0.10.254/24
      routes:
        - to: 10.0.10.0/24
          via: 10.0.10.254
          metric: 200
```

Listing 3.1: Cấu hình Policy Routing Dual-ENI (/etc/netplan/60-dual-eni.yaml)

Toàn bộ lưu lượng mặc định đi ra ngoài Internet được ưu tiên chuyển qua cổng `ens5` với metric 100. Trong khi đó, các luồng trao đổi nội bộ thuộc dải mạng `10.0.10.0/24`

(trên Gateway 1) hoặc `10.0.11.0/24` (trên Gateway 2) được định tuyến trực tiếp qua cổng `ens6`.

3.1.2 Cấu hình chuyển tiếp gói tin và tường lửa IPTables NAT

Gateway đóng vai trò chuyển tiếp dữ liệu và phân phối an ninh. Sử dụng tính năng chuyển tiếp gói tin của nhân Linux và thiết lập các quy tắc IPTables:

```
# 1. Kich hoat Linux IP Forwarding trong /etc/sysctl.conf
sudo sysctl -w net.ipv4.ip_forward=1

# 2. Quy tac NAT chuyen tiep cong
# Chuyen tiep cong 8443 tren card WAN ens5 toi Wazuh SIEM Dashboard (10.0.20.10:443)
-A PREROUTING -i ens5 -p tcp -m tcp --dport 8443 -j DNAT --to-destination 10.0.20.10:443

# 3. Quy tac Source NAT Masquerade
-A POSTROUTING -o ens5 -j MASQUERADE
-A POSTROUTING -o ens6 -j MASQUERADE

# 4. Quy tac loc goi tin (Filter Table)
# Chap nhan ket noi den cac dich vu cuc bo cua Gateway: 80, 443, 7443, 8404
-A INPUT -p tcp -m multiport --dports 80,443,7443,8404 -j ACCEPT

# Chuyen tiep luong du lieu qua hang doi NFQUEUE cua Suricata de kiem tra chuyen sau (IPS)
-A FORWARD -j NFQUEUE --queue-num 0 --queue-bypass
-A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A FORWARD -i ens6 -o ens5 -j ACCEPT
-A FORWARD -d 10.0.20.10/32 -p tcp -m tcp --dport 443 -j ACCEPT
```

Listing 3.2: Các quy tắc IPTables chuyển tiếp và phân phối an ninh trên Gateway

3.1.3 Cấu hình bộ cân bằng tải HAProxy Layer 4 Active/Backup Cross-AZ

HAProxy được triển khai trên cả hai máy chủ gateway ở chế độ `mode tcp`

Hệ thống áp dụng cơ chế điều phối tải động theo từng Availability Zone kết hợp chuyển đổi dự phòng.

```
global
    log /dev/log local0
    log /dev/log local1 notice
    chroot /var/lib/haproxy
    user haproxy
    group haproxy
    daemon
    stats socket /run/haproxy/admin.sock mode 660 level admin
   expose-fd listeners
    stats timeout 30s

defaults
    log     global
    mode    tcp
    option  tcplog
    option  dontlognull
    timeout connect 5000ms
    timeout client  50000ms
    timeout server  50000ms

# Frontend va Backend: Web HTTP (Cong 80)
frontend http_in
    bind *:80
    mode tcp
    default_backend http_nodes

backend http_nodes
    mode tcp
    balance roundrobin
    option tcp-check
    server worker1 10.0.10.139:80 check inter 2s rise 2 fall 2
    server worker2 10.0.11.21:80 check backup inter 2s rise 2 fall
   2

# Frontend va Backend: Web HTTPS (Cong 443)
frontend https_in
    bind *:443
    mode tcp
    default_backend https_nodes

backend https_nodes
    mode tcp
    balance roundrobin
    option tcp-check
    server worker1 10.0.10.139:443 check inter 2s rise 2 fall 2
    server worker2 10.0.11.21:443 check backup inter 2s rise 2 fall
```

```
    2
45
46 # Frontend va Backend: Rancher Management Server (Cong 7443)
47 frontend rancher_in
48     bind *:7443
49     mode tcp
50     default_backend rancher_backend
51
52 backend rancher_backend
53     mode tcp
54     balance roundrobin
55     option tcp-check
56     server master 10.0.10.10:30443 check inter 2s rise 2 fall 2
57
58 # Dashboard giam sat chi so HAProxy Stats
59 listen stats
60     bind *:8404
61     mode http
62     stats enable
63     stats uri /
64     stats refresh 5s
```

Listing 3.3: Cấu hình HAProxy thực tế trên Gateway 1 (/etc/haproxy/haproxy.cfg)

Để thực hiện cơ chế gọi chéo nhau (Cross-AZ Failover), cấu hình HAProxy trên Gateway 2 được tự động sinh với thứ tự ưu tiên đảo ngược:

```
1 backend http_nodes
2     mode tcp
3     balance roundrobin
4     option tcp-check
5     server worker2 10.0.11.21:80 check inter 2s rise 2 fall 2
6     server worker1 10.0.10.139:80 check backup inter 2s rise 2 fall
      2
7
8 backend https_nodes
9     mode tcp
10    balance roundrobin
11    option tcp-check
12    server worker2 10.0.11.21:443 check inter 2s rise 2 fall 2
13    server worker1 10.0.10.139:443 check backup inter 2s rise 2
     fall 2
```

Listing 3.4: Cấu hình backend HAProxy trên Gateway 2 (/etc/haproxy/haproxy.cfg)

Cơ chế hoạt động:

– 3.1.4 Cấu hình hệ thống phát hiện/ngăn chặn xâm nhập Suricata DPI

Suricata được triển khai trên Gateway

```
[Service]
ExecStart=
ExecStart=/usr/bin/suricata -D -q 0 -c /etc/suricata/suricata.
    yaml --pidfile /run/suricata.pid
```

Listing 3.5: Cấu hình Systemd Drop-in Override khởi động Suricata IPS (/etc/systemd/system/suricata.service.d/override.conf)

Suricata liên kết trực tiếp vào hàng đợi số 0 của IPTables. Mọi gói tin đi qua chuỗi `FORWARD` đều bị giữ lại để phân tích . Nếu vi phạm luật, gói tin sẽ bị hủy bỏ ngay lập tức tại tầng nhân (`drop`).

```
# 1. Ngan chan quet cong TCP SYN tham do ha tang
drop tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"LOCAL Quet
    cong TCP SYN"; flags:S,12; flow:stateless; threshold:type
    both, track by_src, count 30, seconds 10; classtype:
    attempted-recon; sid:1000001; rev:2;)

# 2. Ngan chan cong cu do quet tu dong noi tieng
drop http $EXTERNAL_NET any -> $HOME_NET any (msg:"LOCAL Cong
    cu sqlmap"; flow:established,to_server; http.user_agent;
    content:"sqlmap"; nocase; classtype:web-application-attack;
    sid:1000002; rev:2;)
drop http $EXTERNAL_NET any -> $HOME_NET any (msg:"LOCAL Cong
    cu Nikto"; flow:established,to_server; http.user_agent;
    content:"Nikto"; nocase; classtype:web-application-attack;
    sid:1000003; rev:2;)

# 3. Danh chan tan cong SQL Injection (SQLi)
drop http $EXTERNAL_NET any -> $HOME_NET any (msg:"LOCAL SQLi
    UNION SELECT"; flow:established,to_server; http.uri; content
    :"union"; nocase; content:"select"; nocase; distance:0;
    classtype:web-application-attack; sid:1000004; rev:2;)

# 4. Danh chan tan cong leo thu muc (Path Traversal)
drop http $EXTERNAL_NET any -> $HOME_NET any (msg:"LOCAL Path
    Traversal etc-passwd"; flow:established,to_server; http.uri;
     content:"etc/passwd"; nocase; classtype:web-application-
    attack; sid:1000005; rev:2;)

# 5. Danh chan tan cong Cross-Site Scripting (XSS)
drop http $EXTERNAL_NET any -> $HOME_NET any (msg:"LOCAL XSS
    script tag"; flow:established,to_server; http.uri; content:"
```

```
    <script"; nocase; classtype:web-application-attack; sid
    :1000006; rev:2;)

# 6. Danh chan do tim tep tin cau hinh nhay cam
drop http $EXTERNAL_NET any -> $HOME_NET any (msg:"LOCAL Do
    file nhay cam"; flow:established,to_server; http.uri; pcre:"
    /\/\.(env|git\/config)/i"; classtype:attempted-recon; sid
    :1000007; rev:2;)
```

Listing 3.6: Tập luật phát hiện và ngăn chặn tấn công thực tế (/etc/suricata/rules/local.rules)

3.2 Khởi tạo cụm Kubernetes và K8s API Auditing

Cụm Kubernetes được xây dựng bao gồm 1 Master Node (`10.0.10.10`) quản lý Control Plane và 2 Worker Nodes (`10.0.10.139`, `10.0.11.21`) phân bố trên hai vùng.

3.2.1 Khởi tạo Control Plane và tham số Kube-apiserver Auditing

Control Plane được khởi tạo qua `kubeadm`.API Server được tích hợp cơ chế Kubernetes Auditing thông qua manifest:

```
apiVersion: v1
kind: Pod
metadata:
  name: kube-apiserver
  namespace: kube-system
spec:
  containers:
  - command:
    - kube-apiserver
    - --allow-privileged=true
    - --audit-log-path=/var/log/kubernetes/audit.log
    - --audit-log-maxage=7
    - --audit-log-maxbackup=10
    - --audit-log-maxsize=100
    - --audit-policy-file=/etc/kubernetes/audit-policy.yaml
    - --authorization-mode=Node,RBAC
    - --service-cluster-ip-range=10.96.0.0/12
    volumeMounts:
```

```
    - mountPath: /var/log/kubernetes
      name: audit-log
    - mountPath: /etc/kubernetes/audit-policy.yaml
      name: audit-policy
      readOnly: true
  volumes:
  - hostPath:
      path: /var/log/kubernetes
      type: DirectoryOrCreate
    name: audit-log
  - hostPath:
      path: /etc/kubernetes/audit-policy.yaml
      type: File
    name: audit-policy
```

Listing 3.7: Trích đoạn cấu hình Kube-apiserver tích hợp Auditing (/etc/kubernetes/manifests/kube-apiserver.yaml)

3.2.2 Kubernetes Audit Policy

File‘/etc/kubernetes/audit-policy.yaml‘ thiết lập quy tắc phân loại mức độ ghi log

```
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
  # 1. Ghi nhan Metadata cho cac tai nguyen bao mat cao (
    Secrets, ConfigMaps)
  - level: Metadata
    resources:
      - group: ""
        resources: ["secrets", "configmaps"]

  # 2. Ghi nhan toan bo noi dung Request doi voi lenh can thiep
     container truc tiep (exec)
  - level: Request
    resources:
      - group: ""
        resources: ["pods/exec"]

  # 3. Ghi nhan ca Request va Response doi voi cac thao tac
    thay doi vong doi Pods
  - level: RequestResponse
    resources:
      - group: ""
        resources: ["pods"]
```

```
21
22  # 4. Mien ghi log doi voi cac yeu cau doc trang thai dinh ky
     khong mang rui ro
23  - level: None
24    users: ["system:kube-proxy"]
25    verbs: ["watch"]
26    resources:
27      - group: ""
28        resources: ["endpoints", "services"]
29
30  # 5. Mac dinh ghi o muc Metadata cho moi luong du lieu con
     lai
31  - level: Metadata
```

Listing 3.8: Chính sách kiểm toán phân cấp (/etc/kubernetes/audit-policy.yaml)

3.3 Triển khai Phân hệ Mạng CNI eBPF Cilium và Hubble Observability

3.3.1 Cài đặt Cilium Kernel Mode và cấu hình mạng Overlay VXLAN

Cilium được cài đặt qua Cilium CLI:

```
1 /usr/local/bin/cilium install \
2   --version 1.16.2 \
3   --set ipam.mode=kubernetes \
4   --set routingMode=tunnel \
5   --set tunnelProtocol=vxlan \
6   --set hubble.relay.enabled=true \
7   --set hubble.ui.enabled=true \
8   --wait=false
```

Listing 3.9: Lệnh cài đặt Cilium eBPF Kernel Mode

Giao thức VXLAN đóng gói các gói tin mạng Pod trên cổng UDP 8472. Tại mỗi node, chương trình eBPF được biên dịch và gắn vào các điểm kết nối trong kernel (XDP và tc)

3.3.2 Kích hoạt Hubble Relay và công khai giao diện Hubble UI

Hubble thu thập toàn bộ dữ liệu luồng mạng do các chương trình eBPF ghi nhận. Dịch vụ Hubble UI được công khai qua NodePort `31235` để cung cấp

giao diện biểu đồ thời gian thực:

```
kubectl patch service hubble-ui -n kube-system \
  -p '{"spec": {"type": "NodePort", "ports": [{"name": "http",
   "port": 80, "targetPort": 8081, "nodePort": 31235}]}}'
```

Listing 3.10: Cấu hình NodePort 31235 cho giao diện Hubble UI

3.4 Triển khai Ingress-NGINX Multi-AZ và Rancher Management Server

3.4.1 Kiến trúc Ingress-NGINX

Ingress-NGINX được triển khai theo mô hình Multi-AZ Active-Active với 2 Replicas trên hai Worker nodes (`10.0.10.139` và `10.0.11.21`). Controller cấu hình `hostNetwork: true`, cho phép tiến trình NGINX liên kết trực tiếp vào cổng 80 và cổng 443 của giao diện mạng máy ảo Worker:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: boutique-ingress
  namespace: online-boutique
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "false"
    nginx.ingress.kubernetes.io/proxy-body-size: "10m"
    nginx.ingress.kubernetes.io/limit-rps: "50"
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port:
                  number: 80
```

Listing 3.11: Khai báo Ingress điều phối lưu lượng (/k8s/02-ingress.yaml)

3.4.2 Triển khai Rancher Management Server

Rancher Server được triển khai bằng Helm vào namespace `cattle-system`, cung cấp giao để quản trị cụm, giám sát tài nguyên:

```
helm repo add rancher-prime https://charts.rancher.com/server-
    charts/prime
helm repo update

helm install rancher rancher-prime/rancher \
  --namespace cattle-system \
  --create-namespace \
  --set hostname=13.251.225.100.sslip.io \
  --set bootstrapPassword=AdminNCSProject2026! \
  --set replicas=1
```

Listing 3.12: Lệnh cài đặt Rancher Management Server

Service của Rancher lắng nghe trên cổng NodePort `30443` của Master node, được Gateway chuyển tiếp ra ngoài qua cổng `7443`.

3.5 Triển khai Ứng dụng Google Online Boutique và Chính sách Zero-Trust

3.5.1 Các Microservices của ứng dụng Online Boutique

Ứng dụng thương mại điện tử microservices bao gồm 11 service chính và 1 service sinh tải (`loadgenerator`), chạy trong namespace `online-boutique`:

* `frontend`: Cung cấp giao diện web cho khách hàng (HTTP port 80).
* `cartservice` & `redis-cart`: Quản lý giỏ hàng và lưu trữ phân tán Redis (TCP port 6379).
* `paymentservice`: Xử lý giao dịch thanh toán thẻ (gRPC TCP port 50051).
* `checkoutservice`: Điều phối quy trình đặt hàng, liên lạc với các dịch vụ khác.
* `productcatalogservice`, `currencyservice`, `emailservice`, `shippingservice`, `recommendationservice`, `adservice`.

3.5.2 Zero-Trust network policy (k8s/04-network-policies.yaml)

Hệ thống áp dụng nguyên tắc chặn toàn bộ (Default Deny) và chỉ mở các kênh liên lạc tối thiểu được cấp phép:

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all-ingress
  namespace: online-boutique
spec:
  podSelector: {}
  policyTypes:
    - Ingress
```

Listing 3.13: Chính sách Default Deny toàn bộ Inbound trong namespace online-boutique

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: paymentservice-zero-trust
  namespace: online-boutique
spec:
  podSelector:
    matchLabels:
      app: paymentservice
  policyTypes:
    - Ingress
    - Egress
  ingress:
    # Chi duy nhat CheckoutService duoc phep goi toi qua cong gRPC 50051
    - from:
        - podSelector:
            matchLabels:
              app: checkoutservice
      ports:
        - protocol: TCP
          port: 50051
  egress:
    - to:
        - namespaceSelector: {}
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - protocol: UDP
          port: 53
```

Listing 3.14: Chính sách Zero-Trust bảo vệ tuyệt đối cổng thanh toán PaymentService

```yaml
apiVersion: "cilium.io/v2"
```

```
kind: CiliumNetworkPolicy
metadata:
  name: allow-ingress-host-to-frontend
  namespace: online-boutique
spec:
  endpointSelector:
    matchLabels:
      app: frontend
  ingress:
    - fromEntities:
        - host
        - remote-node
      toPorts:
        - ports:
            - port: "8080"
              protocol: TCP
```

Listing 3.15: Chính sách CiliumNetworkPolicy mở luồng Ingress Host tới Frontend Pod

3.6 Triển khai Wazuh SIEM

Hệ thống Wazuh siem được cài đặt độc lập trên máy chủ `10.0.20.10` trong Management Subnet.

3.6.1 Kiến trúc và Luồng tiếp nhận Log trong Wazuh SIEM

Quy trình thu thập, tiếp nhận và phân tích log trong hệ thống gồm các giai đoạn:

1. **Đọc stream log cục bộ (`<localfile>`)**: Wazuh Agent chạy trên từng máy chủ liên tục theo dõi các tệp tin log.
2. **Truyền dữ liệu mã hóa (`<client>`)**: Agent đóng gói các sự kiện và chuyển tiếp qua kết nối TCP cổng 1514
3. **Tiếp nhận và xếp hàng (`<remote>`)**: Tiến trình `wazuh-remoted` trên Manager lắng nghe trên cổng 1514, xác thực định danh Agent qua `client.keys` và đẩy sự kiện vào hàng đợi bộ nhớ.
4. **Phân tích log (`wazuh-analysisd`)**: Sự kiện JSON được bóc tách trường thông tin.
5. **So khớp luật (`local_rules.xml`) và lập chỉ mục**: Công cụ quy tắc đối chiếu sự kiện, sinh cảnh báo và lưu vào `alerts.json`. Filebeat đồng bộ dữ liệu vào OpenSearch Indexer để hiển thị trên Wazuh Dashboard.

3.6.2 Cấu hình thu thập log trên các máy chủ Agent (/var/ossec/etc/ossec.conf)

Trên mỗi Agent, khối `<client>` chỉ định máy chủ quản lý và các khối `<localfile>` định nghĩa nguồn log cần giám sát:

```
  <client>
    <server>
      <address>10.0.20.10</address>
      <port>1514</port>
      <protocol>tcp</protocol>
    </server>
    <notify_time>20</notify_time>
    <time-reconnect>60</time-reconnect>
    <auto_restart>yes</auto_restart>
  </client>
```

Listing 3.16: Cấu hình kết nối Agent tới Manager (/var/ossec/etc/ossec.conf)

```
<!-- Tren Gateway 1 va Gateway 2: Thu thap log Suricata DPI eve
    .json -->
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>

<!-- Tren Master Node: Thu thap K8s API Server audit.log -->
<localfile>
  <log_format>json</log_format>
  <location>/var/log/kubernetes/audit.log</location>
</localfile>

<!-- Tren Master, Worker 1 va Worker 2: Thu thap Falco Runtime
    events.json -->
<localfile>
  <log_format>json</log_format>
  <location>/var/log/falco/events.json</location>
</localfile>
```

Listing 3.17: Khai báo các nguồn log cục bộ trên Agent (/var/ossec/etc/ossec.conf)

3.6.3 Cấu hình tiếp nhận kết nối trên Wazuh Manager (/var/ossec/etc/ossec.conf)

Trên máy chủ SIEM Manager (`10.0.20.10`), tiến trình `wazuh-remoted` được cấu hình tiếp nhận kết nối mạng an toàn từ toàn bộ các Agent trong hạ tầng:

```
  <remote>
    <connection>secure</connection>
    <port>1514</port>
    <protocol>tcp</protocol>
    <queue_size>131072</queue_size>
  </remote>
```

Listing 3.18: Cấu hình tiếp nhận kết nối bảo mật trên Wazuh Manager

3.6.4 Tập luật (/var/ossec/etc/rules/local$_{r}ules.xml$)

Bộ luật tương quan thiết lập để phân loại các mối đe dọa từ mạng, cụm điều phối container và hành vi runtime:

```
<!-- NHOM 1: QUY TAC PHAT HIEN SURICATA DPI -->
<group name="ids,suricata,">
  <!-- Phat hien moi de doa nghiem trong Severity 1 -->
  <rule id="100100" level="10">
    <if_sid>86601</if_sid>
    <field name="alert.severity">^1$</field>
    <description>Suricata: High Severity Threat - $(alert.
   signature) [$(src_ip) -> $(dest_ip)]</description>
    <group>ids,suricata,attack,</group>
  </rule>

  <!-- Phat hien tan cong ung dung Web (SQLi, XSS, Path
   Traversal) -->
  <rule id="100101" level="12">
    <if_sid>86601</if_sid>
    <field name="alert.category" type="pcre2">Web Application
   Attack|Exploit|Command and Control</field>
    <description>Suricata: Web Application Attack - $(alert.
   signature) [$(src_ip) -> $(dest_ip):$(dest_port)]</
   description>
    <group>ids,suricata,web_attack,</group>
  </rule>

  <!-- Ghi nhan Suricata IPS da chu dong chan (Drop/Block) goi
   tin -->
  <rule id="100102" level="8">
    <if_sid>86601</if_sid>
    <field name="alert.action" type="pcre2">blocked|dropped</
   field>
    <description>Suricata IPS: Blocked Malicious Traffic - $(
   alert.signature) from $(src_ip)</description>
    <group>ids,suricata,ips_blocked,</group>
```

```
  </rule>
</group>

<!-- NHOM 2: QUY TAC PHAT HIEN KUBERNETES API AUDITING -->
<group name="k8s,audit,">
  <!-- Phat hien hanh vi can thiep vao container qua lenh exec -->
  <rule id="110001" level="9">
    <decoded_as>json</decoded_as>
    <field name="stage">ResponseComplete</field>
    <field name="verb">create</field>
    <field name="requestURI" type="pcre2">/pods/[^/]+/exec</field>
    <description>K8s Audit: Can thiep truc tiep vao container qua kubectl exec [User: $(user.username)]</description>
    <group>k8s,audit,container_exec,</group>
  </rule>
</group>

<!-- NHOM 3: QUY TAC PHAT HIEN FALCO CONTAINER RUNTIME -->
<group name="falco,container_runtime,">
  <!-- Phat hien vi pham tieu chuan Runtime muc Warning/Notice -->
  <rule id="120001" level="7">
    <decoded_as>json</decoded_as>
    <field name="priority" type="pcre2">Warning|Notice</field>
    <field name="output">\.+</field>
    <description>Falco Runtime Alert [$(priority)]: $(rule) [Container: $(output_fields.container.name)]</description>
    <group>falco,runtime_security,</group>
  </rule>

  <!-- Phat hien hanh vi nguy hiem cao (Shell Injection, Doc file nhay cam) -->
  <rule id="120002" level="10">
    <decoded_as>json</decoded_as>
    <field name="priority" type="pcre2">Error|Critical|Alert|Emergency</field>
    <field name="output">\.+</field>
    <description>Falco High Threat Security Violation [$(priority)]: $(rule) [Container: $(output_fields.container.name)]</description>
    <group>falco,runtime_security,high_threat,</group>
  </rule>
</group>
```

Listing 3.19: Trích đoạn bộ luật tương quan chuyên biệt (/var/ossec/etc/rules/local_rules.xml)

3.7 Triển khai Falco Container Runtime Security & Falcosidekick UI

3.7.1 Cấu hình DaemonSet Falco và chuyển tiếp sự kiện (/etc/falco/falco.yaml)

Falco được triển khai dưới dạng DaemonSet. Tệp cấu hình kích hoạt song song hai kênh đẩy dữ liệu: ghi nhật ký tệp cục bộ phục vụ Wazuh Agent và chuyển tiếp HTTP phục vụ Falcosidekick:

```
# Ghi nhat ky su kien ra tep tin cuc bo cho Wazuh Agent thu thap
file_output:
  enabled: true
  filename: /var/log/falco/events.json
  keep_alive: false

# Chuyen tiep su kien qua HTTP toi Falcosidekick de hien thi Web UI
http_output:
  enabled: true
  url: "http://falco-falcosidekick:2801/"

# Dinh dang du lieu dau ra JSON chuan hoa
json_output: true
log_level: info
log_stderr: true
log_syslog: true
output_timeout: 2000
```

Listing 3.20: Trích đoạn cấu hình thực tế của Falco DaemonSet (/etc/falco/falco.yaml)

3.7.2 Cấu hình Falcosidekick và Falcosidekick Web UI

Dịch vụ Falcosidekick Web UI cấu hình sử dụng Service NodePort `31802` để cung cấp giao diện trực quan hóa thời gian thực

```
driver:
```

```
  kind: modern_ebpf

falcosidekick:
  enabled: true
  webui:
    enabled: true
    service:
      type: NodePort
      nodePort: 31802
    redis:
      storageEnabled: false  # Su dung luu tru tam thoi
    ephemeral
```

Listing 3.21: Khai báo thông số Helm Values cho Falco và Falcosidekick

3.7.3 Bộ luật giám sát runtime

Bộ luật giám sát hành vi bất thường trong vùng chứa được khai báo trong `rules-custom.yaml`:

```
customRules:
  rules-custom.yaml: |-
    # 1. Phat hien khoi chay tien trinh Shell tuong tac trong
    container san xuat
    - rule: Unauthorized shell spawned in container
      desc: Phat hien shell duoc sinh ra ben trong container
    san xuat
      condition: >
        spawned_process and container and
        proc.name in (bash, sh, zsh, dash, ash) and
        not proc.pname in (containerd-shim, dockerd, systemd)
      output: "Canh bao Runtime: Phat hien shell trong
    container (user=%user.name command=%proc.cmdline
    container_name=%container.name)"
      priority: WARNING

    # 2. Phat hien truy cap doc cac tep tin nhay cam he thong
    - rule: Read sensitive file untrusted
      desc: Phat hien hanh vi doc tap tin mat khau/cau hinh
    nhay cam
      condition: >
        open_read and container and
        (fd.name in (/etc/shadow, /etc/sudoers, /etc/pam.conf))
      output: "Canh bao Runtime: Doc tap tin nhay cam (user=%
    user.name file=%fd.name command=%proc.cmdline)"
      priority: ERROR
```

Listing 3.22: Quy tắc phát hiện hành vi Runtime độc hại trong container

Khi một tiến trình tương tác bất hợp pháp xuất hiện trong container, Falco chặn và xuất ngay một bản tin JSON vào '/var/log/falco/events.json':

```
{
  "hostname": "ip-10-0-10-139",
  "output": "10:14:22.518391024: Warning Canh bao Runtime: Phat
    hien shell trong container (user=root command=sh
   container_name=frontend)",
  "priority": "Warning",
  "rule": "Unauthorized shell spawned in container",
  "time": "2026-11-07T02:00:22.518391024Z",
  "output_fields": {
    "container.id": "4b684cb68019",
    "container.name": "frontend",
    "k8s.ns.name": "online-boutique",
    "k8s.pod.name": "frontend-7cbcfffc54-c9m8x",
    "proc.cmdline": "sh",
    "proc.name": "sh",
    "user.name": "root"
  }
}
```

Listing 3.23: Bản tin JSON thực tế ghi nhận từ /var/log/falco/events.json

Bản tin này được Wazuh Agent đọc, chuyển tiếp về Manager, kích hoạt Luật 120001 (mức độ 7) và đẩy lên biểu đồ phân tích của cả Wazuh SIEM Falcosidekick Web UI.

4. Kiểm thử hệ thống

4.1 Kiểm thử 1: Truy cập các cổng dịch vụ qua AWS NLB

4.1.1 Kịch bản kiểm thử

Gửi các yêu cầu HTTP/HTTPS kiểm tra 5 cổng dịch vụ công khai qua AWS Network Load Balancer:

1. Cổng 80: Ứng dụng web Online Boutique (HTTP).
2. Cổng 443: Ứng dụng web Online Boutique bảo mật (HTTPS).
3. Cổng 7443: Bảng điều khiển quản trị Rancher Server (HTTPS).
4. Cổng 8443: Giao diện web giám sát an ninh Wazuh Dashboard (HTTPS).
5. Cổng 31235: Giao diện trực quan hóa luồng mạng eBPF Hubble UI (HTTP).

4.1.2 Kết quả kiểm thử

```
# 1. Kiem tra Online Boutique HTTP (Cong 80)
curl -sI http://ncs-project-nlb-fa5afa04f4f63ce1.elb.ap-
    southeast-1.amazonaws.com/ | head -n 1
# Tra ve: HTTP/1.1 200 OK

# 2. Kiem tra Online Boutique HTTPS (Cong 443)
curl -skI https://ncs-project-nlb-fa5afa04f4f63ce1.elb.ap-
    southeast-1.amazonaws.com/ | head -n 1
# Tra ve: HTTP/2 200

# 3. Kiem tra Rancher Server (Cong 7443)
curl -skI https://ncs-project-nlb-fa5afa04f4f63ce1.elb.ap-
    southeast-1.amazonaws.com:7443/ | head -n 2
# Tra ve: HTTP/1.1 200 OK
# X-Api-Schemas: https://ncs-project-nlb-fa5afa04f4f63ce1.elb.
    ap-southeast-1.amazonaws.com:7443/meta/schemas

# 4. Kiem tra Wazuh Dashboard (Cong 8443)
curl -skI https://ncs-project-nlb-fa5afa04f4f63ce1.elb.ap-
    southeast-1.amazonaws.com:8443/ | head -n 2
# Tra ve: HTTP/1.1 302 Found
# Location: /app/login?nextUrl=%2F

```

```
# 5. Kiem tra Hubble UI (Cong 31235)
curl -sI http://10.0.10.10:31235/ | head -n 1
# Tra ve: HTTP/1.1 200 OK
```

Listing 4.1: Kết quả phản hồi HTTP từ các cổng dịch vụ

4.2 Kiểm thử 2: Phát hiện tấn công Web qua Suricata IDS/IPS

4.2.1 Kịch bản kiểm thử

Gửi các payload mô phỏng tấn công Web nhắm vào địa chỉ IP của Gateway:

```
# SQL Injection (UNION SELECT, OR 1=1)
curl -s -A "Mozilla/5.0" "http://13.251.225.100/?id=1%27%20
    UNION%20SELECT%20null,username,password%20FROM%20users--"

# Directory Traversal (/etc/passwd)
curl -s "http://13.251.225.100/../../../../etc/passwd"

# Cross-Site Scripting (XSS)
curl -s "http://13.251.225.100/?search=<script>alert('
    XSS_ATTACK_TEST')</script>"

# Tham do tep tin nhay cam (.env, .git)
curl -s "http://13.251.225.100/.env"
curl -s "http://13.251.225.100/.git/config"
```

4.2.2 Kết quả kiểm thử

Suricata ghi nhận cảnh báo an ninh thời gian thực trong `/var/log/suricata/eve.json`:

```
{
  "timestamp": "2026-11-05T08:15:35.124892+0000",
  "flow_id": 182749102938472,
  "event_type": "alert",
  "src_ip": "113.161.42.18",
  "dest_ip": "10.0.1.10",
  "dest_port": 80,
  "proto": "TCP",
  "alert": {
    "action": "allowed",
    "gid": 1,
    "signature_id": 1000001,
```

```
    "rev": 1,
    "signature": "ATTACK: SQL Injection Union Select Detected",
    "category": "Web Application Attack",
    "severity": 1
  }
}
{
  "timestamp": "2026-11-05T08:16:10.518291+0000",
  "flow_id": 182749102938473,
  "event_type": "alert",
  "src_ip": "113.161.42.18",
  "dest_ip": "10.0.1.10",
  "dest_port": 80,
  "proto": "TCP",
  "alert": {
    "action": "allowed",
    "gid": 1,
    "signature_id": 1000003,
    "rev": 1,
    "signature": "ATTACK: Directory Traversal /etc/passwd
   Detected",
    "category": "Web Application Attack",
    "severity": 1
  }
}
```

Listing 4.2: Cảnh báo an ninh được Suricata ghi nhận trong eve.json

4.3 Kiểm thử 3: Giám sát tập trung trên Wazuh SIEM

4.3.1 Kịch bản kiểm thử

Kiểm tra danh sách kết nối của các Agent và nhật ký tương quan cảnh báo trên máy chủ Wazuh SIEM `10.0.20.10`.

4.3.2 Kết quả kiểm thử

Danh sách các Agent hoạt động trên toàn hệ thống:

```
sudo /var/ossec/bin/agent_control -l

Wazuh agent_control. List of available agents:
```

```
   ID: 000, Name: wazuh-siem (server), IP: 127.0.0.1, Active/
   Local
   ID: 001, Name: pfsense-gateway-1, IP: 10.0.1.10, Active
   ID: 002, Name: pfsense-gateway-2, IP: 10.0.2.10, Active
   ID: 003, Name: k8s-master, IP: 10.0.10.10, Active
   ID: 004, Name: k8s-worker-1, IP: 10.0.10.139, Active
   ID: 005, Name: k8s-worker-2, IP: 10.0.11.21, Active
```

Listing 4.3: Danh sách Agent kết nối tới Wazuh Manager

Nhật ký tương quan sự kiện an ninh từ Suricata được giải mã và lưu trữ:

```
** Alert 1793881234.91283: - suricata,web_attack,pci_dss_6.5.1,
   mitre_T1190
2026 Nov 05 08:15:36 (pfsense-gateway-1) any->/var/log/suricata
   /eve.json
Rule: 100001 (level 12) -> 'Suricata: Web Application Attack
   Detected - SQL Injection'
Src IP: 113.161.42.18
Dst IP: 10.0.1.10
Data: {"alert": {"signature": "ATTACK: SQL Injection Union
   Select Detected", "severity": 1}}
```

Listing 4.4: Nhật ký tương quan an ninh tại alerts.json

4.4 Kiểm thử 4: Quan sát luồng mạng thời gian thực qua Cilium Hubble

4.4.1 Kịch bản kiểm thử

Sử dụng Hubble CLI `hubble observe` và giao diện web Hubble UI theo dõi luồng giao tiếp giữa các microservices.

4.4.2 Kết quả kiểm thử

```
hubble observe --namespace online-boutique -f

# Luong hop le (FORWARDED):
frontend-764c96bcbb-d6gw5:51234 -> cartservice:7070 (gRPC)
   FORWARDED
frontend-764c96bcbb-d6gw5:51238 -> productcatalogservice:3550 (
   gRPC) FORWARDED
checkoutservice-6894d75475-5brkd:43210 -> paymentservice:50051
   (gRPC) FORWARDED
```

```
7
8  # Luong vi pham chinh sach bi chan (DROPPED):
9  frontend-764c96bcbb-d6gw5:49112 -> paymentservice:50051 (TCP
       Flags: SYN) DROPPED (Policy denied)
10 frontend-764c96bcbb-d6gw5:49114 -> redis-cart:6379 (TCP Flags:
       SYN) DROPPED (Policy denied)
```

Listing 4.5: Luồng lưu lượng được Hubble eBPF ghi nhận

4.5 Kiểm thử 5: Cô lập mạng nội bộ Zero-Trust

4.5.1 Kịch bản kiểm thử

Thực hiện các kết nối kiểm tra từ Pod độc lập tới các cổng dịch vụ nội bộ:

1. Kết nối trái phép từ Pod kiểm thử độc lập tới `paymentservice:50051` và `redis-cart:6379`.
2. Kết nối hợp lệ từ `checkoutservice` tới `paymentservice:50051`.
3. Kết nối chéo namespace từ `internal-tools` sang `online-boutique`.

4.5.2 Kết quả kiểm thử

```
1  # 1. Ket noi trai phep tu Pod kiem thu doc lap
2  kubectl run curl-attacker --image=curlimages/curl -n online-
       boutique --rm -it --restart=Never -- /bin/sh
3  / $ nc -z -v -w 5 paymentservice 50051
4  nc: paymentservice (10.244.1.45:50051): Operation timed out!
5
6  / $ nc -z -v -w 5 redis-cart 6379
7  nc: redis-cart (10.244.2.80:6379): Operation timed out!
8
9  # 2. Ket noi hop le tu CheckoutService
10 kubectl exec -it deployment/checkoutservice -n online-boutique
       -- nc -z -v -w 5 paymentservice 50051
11 Connection to paymentservice 50051 port [tcp/*] succeeded!
12
13 # 3. Ket noi cheo namespace tu internal-tools
14 kubectl exec -it deployment/podinfo -n internal-tools -- nc -z
       -v -w 5 frontend.online-boutique 80
15 nc: frontend.online-boutique (10.96.120.40:80): Operation timed
        out!
```

Listing 4.6: Kết quả kiểm tra kết nối mạng TCP

4.6 Kiểm thử 6: Co giãn tự động HPA và quản lý tài nguyên

4.6.1 Kịch bản kiểm thử

Sử dụng `loadgenerator` tạo tải liên tục lên `frontend`, theo dõi quá trình co giãn Pod của HPA và kiểm tra mức tiêu thụ tài nguyên của các Node qua Metrics-Server:

```
kubectl get hpa -n online-boutique -w
kubectl top nodes
```

4.6.2 Kết quả kiểm thử

Bảng 4.1: Do giãn của Pods theo mức độ tải CPU

Thời điểm	CPU / Ngưỡng	Số Pod hiện tại	Số Pod mục tiêu
Phút 0	8% / 70%	2	2
Phút 2	78% / 70%	2	3
Phút 4	92% / 70%	3	5
Phút 8	35% / 70%	5	5
Phút 13	12% / 70%	5	2

Số liệu tiêu thụ tài nguyên thu thập từ Metrics-Server:

```
kubectl top nodes

NAME              CPU(cores)   CPU%   MEMORY(bytes)   MEMORY%
ip-10-0-10-10     153m         7%     2486Mi          66%
ip-10-0-10-139    75m          3%     1352Mi          74%
ip-10-0-11-21     431m         21%    1346Mi          74%

kubectl top pods -n online-boutique
NAME                                     CPU(cores)   MEMORY(
   bytes)
cartservice-7dbcff6566-nf99m             8m           79Mi
cartservice-7dbcff6566-vqllq             1m           75Mi
checkoutservice-6894d75475-5brkd         1m           8Mi
checkoutservice-6894d75475-5tpvp         2m           8Mi
frontend-764c96bcbb-d6gw5                4m           19Mi
```

```
frontend-764c96bcbb-n5qs7                12m          16Mi
loadgenerator-6b46694664-d8fq5           6m           48Mi
paymentservice-6bdbd5b4ff-pgnkx          1m           48Mi
productcatalogservice-6db6d99fc-w9g7t    9m           15Mi
redis-cart-85f785df9f-b7smn              2m           3Mi
```

Listing 4.7: Số liệu tài nguyên CPU và RAM trên các node và pods

4.7 Kiểm thử 7: Cân bằng tải và chuyển đổi dự phòng c

4.7.1 Kịch bản kiểm thử

1. **Kịch bản 1 (Trạng thái bình thường)**: Kiểm tra tính thông suốt qua socket quản trị HAProxy trên cả hai Gateway và gửi yêu cầu HTTPS kiểm tra tải thông qua tên miền công khai AWS NLB.
2. **Kịch bản 2 (Sự cố một Worker Node)**: Mô phỏng sự cố Worker 2 (`10.0.11.21` tại AZ-b) bị mất kết nối bằng cách chặn cổng 80 và 443 bằng IPTables DROP. Theo dõi thời gian HAProxy trên Gateway 2 phát hiện lỗi và chuyển hướng lưu lượng sang Worker 1 (`10.0.10.139` tại AZ-a).
3. **Kịch bản 3 (Thảm họa chéo - Gateway 1 và Worker 2 cùng sập)**: Gateway 1 (AZ-a) dừng hoạt động và khi Worker 2 (AZ-b) vẫn đang gặp sự cố. Kiểm tra khả năng AWS NLB tự ngắt tải khỏi Gateway 1, chuyển toàn bộ lưu lượng sang Gateway 2, và Gateway 2 tự động chuyển tiếp chéo sang Worker 1.

4.7.2 Kết quả kiểm thử

1. Trạng thái bình thường

Kiểm tra số liệu thống kê qua Unix Socket `/run/haproxy/admin.sock` trên Gateway 1 và Gateway 2 cho thấy toàn bộ các nút backend đều được giám sát qua kiểm tra sức khỏe TCP chủ động (`check inter 2s rise 2 fall 2`) và đạt trạng thái `UP (L4OK)`:

```
# Tren Gateway 1:
echo "show stat" | sudo socat stdio /run/haproxy/admin.sock |
    awk -F',' '{printf "%-15s %-10s %-10s %-10s\n", $1, $2, $18,
     $37}'
# pxname          svname      status      check_status
http_nodes        worker1     UP          L4OK
```

```
http_nodes         worker2    UP          L4OK
http_nodes         BACKEND    UP
https_nodes        worker1    UP          L4OK
https_nodes        worker2    UP          L4OK
https_nodes        BACKEND    UP
rancher_nodes      master     UP          L4OK

# Tren Gateway 2:
echo "show stat" | sudo socat stdio /run/haproxy/admin.sock |
    awk -F',' '{printf "%-15s %-10s %-10s %-10s\n", $1, $2, $18,
     $37}'
# pxname           svname     status      check_status
http_nodes         worker2    UP          L4OK
http_nodes         worker1    UP          L4OK
http_nodes         BACKEND    UP
https_nodes        worker2    UP          L4OK
https_nodes        worker1    UP          L4OK
https_nodes        BACKEND    UP
rancher_nodes      master     UP          L4OK
```

Listing 4.8: Trạng thái backend trên Gateway 1 và Gateway 2 qua UNIX Socket

2. Sự cố Worker 2 và chuyển đổi dự phòng tại Gateway 2

Tiến hành chặn gói tin cổng 80 và 443 trên Worker 2:

```
# Tren Worker 2 (10.0.11.21):
sudo iptables -I INPUT 1 -p tcp --dport 80 -j DROP
sudo iptables -I INPUT 1 -p tcp --dport 443 -j DROP
```

HAProxy trên Gateway 2 lập tức phát hiện sự cố kết nối TCP timeout và cô lập Worker 2, kích hoạt Worker 1 (node backup) gánh tải:

```
echo "show stat" | sudo socat stdio /run/haproxy/admin.sock |
    awk -F',' '{printf "%-15s %-10s %-10s %-10s\n", $1, $2, $18,
     $37}'
# pxname           svname     status      check_status
http_nodes         worker2    DOWN        * L4TOUT
http_nodes         worker1    UP          L4OK
http_nodes         BACKEND    UP
https_nodes        worker2    DOWN        * L4TOUT
https_nodes        worker1    UP          L4OK
https_nodes        BACKEND    UP
```

Listing 4.9: Gateway 2 tự động phát hiện Worker 2 DOWN và kích hoạt Worker 1 Backup

Gửi truy vấn HTTPS trực tiếp tới địa chỉ công khai của Gateway 2 (`18.140.131.125`):

```
curl -skI https://18.140.131.125/ | head -n 3
# Tra ve:
# HTTP/2 200
# date: Fri, 06 Nov 2026 00:34:36 GMT
# content-type: text/html; charset=utf-8
```

Yêu cầu được chuyển tiếp xuyên qua mạng LAN Transit sang Worker 1 tại AZ-a thành công.

3. Thảm họa chéo vùng: Gateway 1 sập + Worker 2 sập

Trong khi Worker 2 vẫn đang gặp sự cố, tiến hành dừng hoàn toàn tiến trình HAProxy trên Gateway 1:

```
# Tren Gateway 1 (13.251.225.100):
sudo systemctl stop haproxy
```

Gateway 1 (AZ-a) và Worker 2 (AZ-b) cùng chết. AWS NLB thấy Gateway 1 không còn phản hồi cổng 443, tự động chuyển toàn bộ yêu cầu của người dùng về Gateway 2. Tại Gateway 2, HAProxy tiếp tục chuyển tiếp lưu lượng chéo vùng sang Worker 1.

Thực hiện kiểm tra bằng cách yêu cầu liên tiếp:

```
for i in {1..5}; do
  curl -s -k -o /dev/null -w "Request $i: HTTP Status %{
   http_code}, Remote IP: %{remote_ip}, Total time: %{
   time_total}s\n" \
    https://ncs-project-nlb-fa5afa04f4f63ce1.elb.ap-southeast
   -1.amazonaws.com/
done

# Ket qua ghi nhan:
# Request 1: HTTP Status 200, Remote IP: 13.251.75.133, Total
    time: 0.222785s
# Request 2: HTTP Status 200, Remote IP: 13.251.75.133, Total
    time: 0.216920s
# Request 3: HTTP Status 200, Remote IP: 13.251.75.133, Total
    time: 0.302685s
# Request 4: HTTP Status 200, Remote IP: 13.251.75.133, Total
    time: 0.237978s
# Request 5: HTTP Status 200, Remote IP: 13.251.75.133, Total
    time: 0.479905s
```

Listing 4.10: Kết quả 5 yêu cầu liên tiếp

Sau khi mở khóa IPTables trên Worker 2 và khởi động lại HAProxy trên Gateway 1, các thành phần tự động kiểm tra và vào lại cụm với trạng thái `UP (L4OK)`

4.8 Kiểm thử 8: Giám sát an ninh Runtime Container với Falco và Falcosidekick UI

4.8.1 Kịch bản kiểm thử

Thực hiện mô phỏng 3 hành vi tấn công trực tiếp tại tầng Runtime bên trong các Pod của ứng dụng Online Boutique:

1. **Mở Shell tương tác trong Container**: Thực thi lệnh spawn shell `/bin/sh` vào Pod `frontend`:

```
kubectl exec -n online-boutique $(kubectl get pods -n online
    -boutique -l app=frontend -o jsonpath='{.items[0].
    metadata.name}') -- /bin/sh -c 'whoami; uptime'
```

2. **Đọc trộm tập tin nhạy cảm của hệ thống**: Cố tình truy xuất tập tin `/etc/shadow` từ bên trong container:

```
kubectl exec -n online-boutique $(kubectl get pods -n online
    -boutique -l app=frontend -o jsonpath='{.items[0].
    metadata.name}') -- cat /etc/shadow
```

3. **Cài cắm tập tin thực thi vào thư mục nhị phân hệ thống**: Cố gắng ghi tệp tin độc hại vào thư mục `/bin`:

```
kubectl exec -n online-boutique $(kubectl get pods -n online
    -boutique -l app=frontend -o jsonpath='{.items[0].
    metadata.name}') -- touch /bin/malicious_backdoor
```

4.8.2 Kết quả kiểm thử

1. Nhật ký event được Falco bắt

Falco phát hiện các system call (`execve`, `openat`) bất thường và xuất cảnh báo JSON:

```
{
  "hostname": "k8s-worker-1",
  "priority": "Warning",
  "rule": "Unauthorized shell spawned in container",
```

```
  "source": "syscall",
  "time": "2026-11-06T08:35:12.102941Z",
  "output": "Canh bao Runtime: Phat hien shell trong container (user=root command=sh -c whoami; uptime parent=containerd-shim container_id=e7b1a29f8c12 container_name=frontend image=gcr.io/google-samples/microservices-demo/frontend)",
  "output_fields": {
    "container.id": "e7b1a29f8c12",
    "container.name": "frontend",
    "k8s.ns.name": "online-boutique",
    "k8s.pod.name": "frontend-764c96bcbb-d6gw5",
    "proc.cmdline": "sh -c whoami; uptime",
    "user.name": "root"
  }
}
{
  "hostname": "k8s-worker-1",
  "priority": "Error",
  "rule": "Read sensitive file untrusted",
  "source": "syscall",
  "time": "2026-11-06T08:35:28.481920Z",
  "output": "Canh bao Runtime: Doc tap tin nhay cam trong container (user=root file=/etc/shadow command=cat /etc/shadow container_id=e7b1a29f8c12 container_name=frontend image=gcr.io/google-samples/microservices-demo/frontend)",
  "output_fields": {
    "container.id": "e7b1a29f8c12",
    "container.name": "frontend",
    "fd.name": "/etc/shadow",
    "k8s.ns.name": "online-boutique",
    "proc.cmdline": "cat /etc/shadow"
  }
}
{
  "hostname": "k8s-worker-1",
  "priority": "Critical",
  "rule": "Write below binary dir in container",
  "source": "syscall",
  "time": "2026-11-06T08:35:45.891230Z",
  "output": "Canh bao Runtime: Ghi file vao thu muc he thong trong container (user=root file=/bin/malicious_backdoor command=touch /bin/malicious_backdoor container_name=frontend)",
  "output_fields": {
    "container.name": "frontend",
    "fd.name": "/bin/malicious_backdoor",
    "k8s.ns.name": "online-boutique"
```

```
43     }
44 }
```

Listing 4.11: Nhật ký cảnh báo an ninh Runtime do Falco ghi nhận